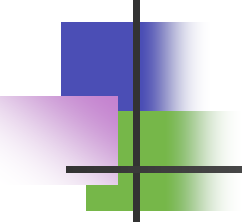


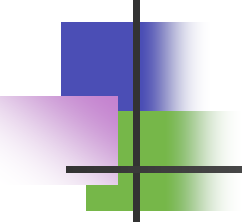
Les Entreprise Java Beans:Processus de Développement, de Deploiement et d'Exécution

Chargé de cours: Dr Yacouba Goita



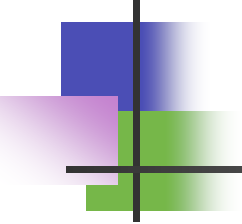
Plan

- ❖ Développement d'un EB
 - Ecrire la classe Primary Key (pour
 - Ecrire la Home interface
 - Ecrire la Remote interface
 - Ecrire l'implémentation du bean
 - Compiler ces classes et interfaces



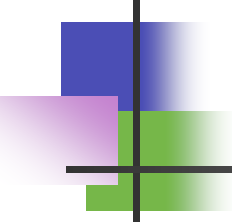
Plan

- Déploiement du EB
 - Construire le descripteur de déploiement (deploytool)
 - Construire le fichier d'archive .ear
 - Vérifier le fichier d'archive



Plan(suite)

- Exécution du EB
 - Activer j2ee
 - Exécuter le client
 - qui peut être une servlet (éventuellement dans un .war déployé)
 - ou une application Java



Exemple:Beans Bancaires

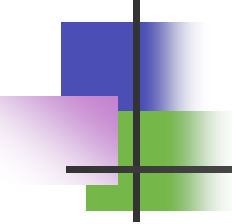
Gestion de comptes bancaires

- Compte Bancaire (Composant persistant ->Entity Bean)
- Interface du Bean(vue par le client)---> Account.java



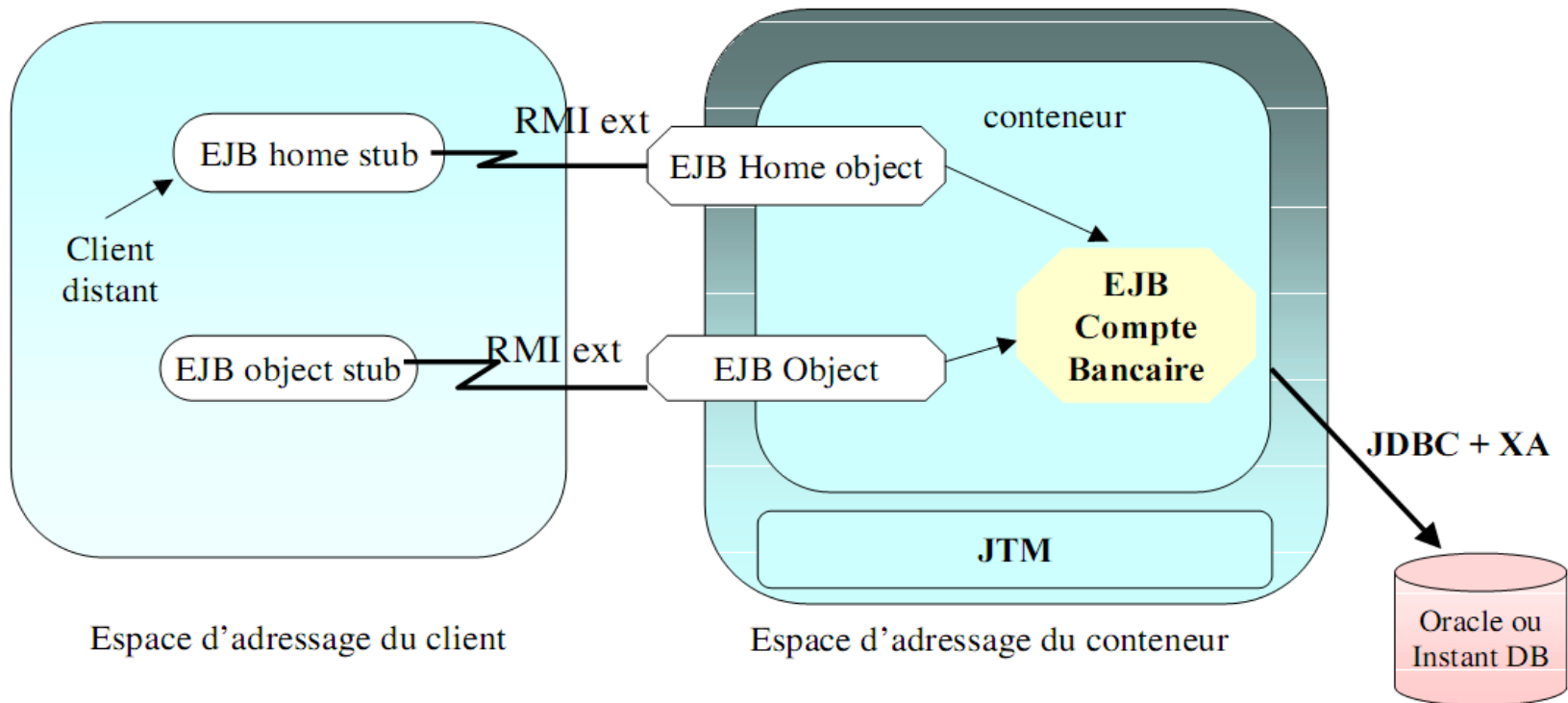
- Interface du Bean(vue par le client) → Account.java

```
public interface Account extends EJBObject {  
    public double getBalance() throws RemoteException;  
    public void setBalance(double d) throws RemoteException;  
    public String getCustomer() throws RemoteException;  
    public void credit(double d) throws RemoteException;  
    public void debit(double d) throws RemoteException;  
}
```

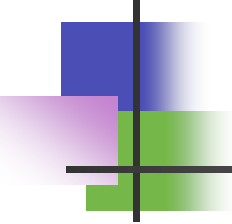
- 
-
- Transfert de Fond (Composant session - Session Bean)
 - Interface du Bean (vue par le client):
Transfer.java:

```
public interface Transfer extends EJBObject {  
    public double transferFund(AccountPK ksrc, AccountPK ktrg,  
                               double d) throws RemoteException;  
    public double transferFund(Account src, Account trg,  
                               double d) throws RemoteException;  
}
```

Architecture de l'application

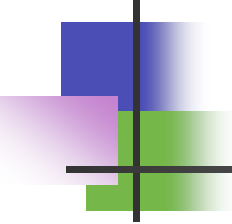


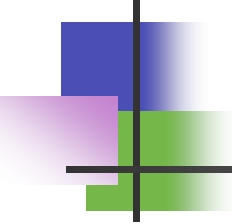
***RMI ext** = extension de RMI afin de propager le contexte de la transaction comme un paramètre supplémentaire (ou utilisation de IIOP)*

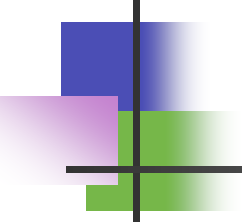


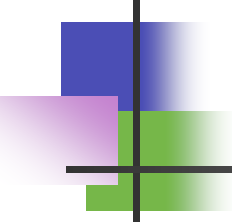
Exemple d 'Entity Bean

- Account
- Spécification du bean
 - entity bean “Account”
 - gestion de la persistance gérée par le conteneur

- 
-
- Le concepteur doit écrire le code suivant
 - • l'interface de gestion du bean - Home interface
 - l'interface d'accès à distance - Remote Interface

- 
-
- La classe pour la gestion des clés d'accès aux différents comptes - Primary Key
 - class (nécessaire uniquement pour les “entity beans”, encapsule le champ représentant la clé primaire d'un “entity bean” dans un objet)

- 
-
- le code du bean
 - • le descripteur pour le déploiement



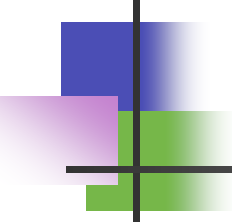
Accountpk.java

- La clé primaire du compte
 - n'est pas toujours nécessaire
 - cas d' une clé primaire non composée
- mais la classe doit implementer l' interface Serializable

```
package Bank;

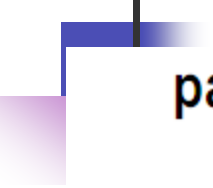
public class AccountPK implements java.io.Serializable {
    public int branchno;
    public int accno;

    public AccountPK(int branchno, int accno) {
        this.branchno = branchno; this.accno = accno; }
    public AccountPK() { }
    public String toString() {
        return String.valueOf(branchno)+"."+ String.valueOf(accno); }
}
```



Accountpk.java:1 'interface distance (remote)

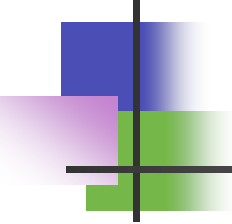
- donne les prototypes des méthodes “ business”
- doit étendre l' interface javax.ejb.EJBObject



```
package Bank;
```

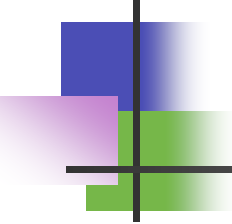
```
public interface Account extends EJBObject {  
    public double getBalance() throws RemoteException;  
    public void setBalance(double d)  
        throws RemoteException, AccountException ;  
    public String getCustomer() throws RemoteException;  
    public void credit(double d)  
        throws RemoteException, AccountException;  
    public void debit(double d)  
        throws RemoteException, AccountException;  
}
```


1 'interface distance (remote)

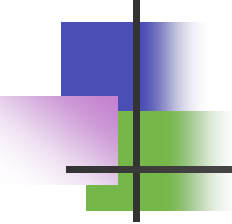


Remarques

- les méthodes setX et getX correspondent à un attribut X
- les paramètres de méthode et les valeurs de retour doivent être de type primitif, serialisable ou étendant/implémentant java.rmi.Remote
- Exceptions
 - les méthodes doivent comporter au minimum RemoteException exception “ système” : SQL, Naming, ...
 - des exceptions applicatives peuvent être ajoutés

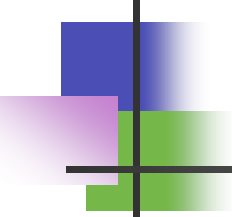


```
package Bank;
public class AccountException extends java.lang.Exception {
    public AccountPK pk;
    public AccountException () { super(); }
    public AccountException (AccountPK pk) { super(); this.pk=pk; }
    public AccountException (msg) { super(msg); } }
```

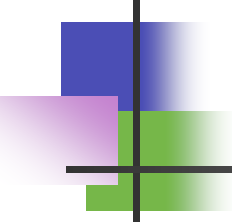


AccountHome.java

- l'interface maison (home)
- donne les prototypes des méthodes
- de création/suppression et de recherche
- équivalent aux Factory de CORBA
- • doit étendre l'interface
`javax.ejb.EJBHome`

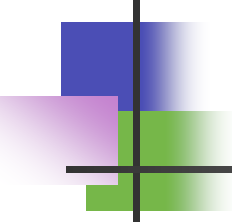


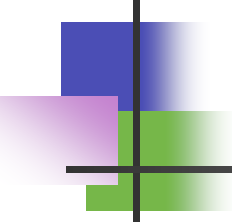
```
package Bank;
public interface AccountHome extends EJBHome {
    public Account create (int branchno, int accno, String customer, double balance)
                        throws RemoteException, CreateException;
    public Account findByPrimaryKey (AccountPK pk)
                        throws RemoteException, FinderException;
    public Enumeration findLargeAccounts (double d)
                        throws RemoteException, FinderException;
    public Enumeration findByCustomer (String str)
                        throws RemoteException, FinderException;
}
```



1 'interface maison (home)

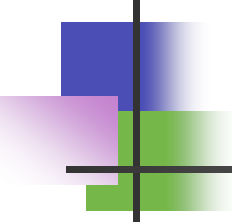
- Remarques sur les méthodes de création
- • comporte `javax.ejb.CreateException`
- • `javax.ejb.DuplicateKeyException` est levée s' il existe déjà une donnée avec la clé primaire

- 
-
- peut ne pas apparaître dans l'interface Home
 - signifie que les données ne peuvent être créées que depuis la source de données (SGBD)
 - par exemple, ordre SQL INSERT

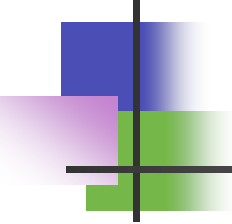


1 'interface maison (home)

- Remarques sur méthode de recherche
 - • comporte `javax.ejb.FinderException`
 - • la méthode de recherche sur clé `findByPrimaryKey` retourne
- un `Account`
- • `javax.ejb.ObjectNotFoundException` est levée si aucun objet n' est trouvé

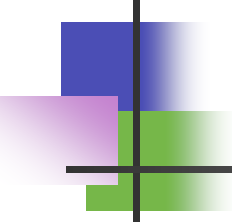
- 
-
- • les autres `findByPrimaryKey` retournent une Enumeration de `Account` ou null si aucun objet n'est trouvé

1 'implantation de 1 'Entity Bean

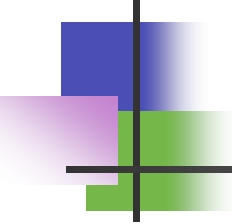


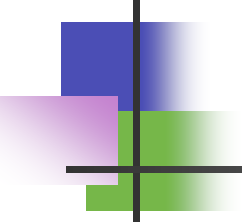
- doit étendre l' interface javax.ejb.EntityBean
- donne l'implantation des méthodes de gestion du cycle de vie (“ callback”)
- ejbActivate() appelé lors de l'activation
- ejbPassivate() appelé lors de la passivation
- ejbLoad() appelé pour charger l'état de l'EB depuis la BD

l'implantation de l'Entity Bean



- `ejbStore()` appelé pour décharger l'état de l'EB vers la BD
- `ejbRemove()` appelé quand le client appelle `remove()`
- `setEntityContext()` appelé par le container quand l'instance est créée
- `unsetEntityContext()` appelé par le container avant de supprimer l'instance

- 
-
- des méthodes “ business” de l’ interface Account
 - getbalance(), setbalance(), ..., credit(), debit() appelées par le client
 - des méthodes de creation de l’ interface AccountHome
 - ejbCreate() ejbPostCreate()

- 
-
- des méthodes de recherche de l'interface AccountHome

`ejbFindByPrimaryKey()` `ejbFindLargeAccounts()`,
`ejbFindByCustomer()`

AccountBean.java

L'implantation de l'Entity Bean

- Les attributs

```
package Bank;
```

```
public class AccountBean implements EntityBean {
```

```
    // les attributs
```

```
    private transient EntityContext ctx;
```

```
    public int      branchno;    // PK
```

```
    public int      accno;       // PK
```

```
    public String   customer;
```

```
    public double   balance;
```

AccountBean.java

1 'implantation de 1 'Entity Bean

- Les méthodes de cycle de vie
- • requises par l' interface EntityBean

```
public void ejbActivate() { }
```

```
public void ejbDestroy() { }
```

```
public void ejbPassivate() { }
```

```
public void ejbLoad() { }
```

```
public void ejbStore() { }
```

```
public void setEntityContext (EntityContext ctx) { this.ctx = ctx; }
```

```
public void unsetEntityContext () { this.ctx = null; }
```

...

--- entreprise ---

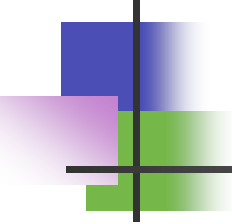
AccountBean.java

1 'implantation de 1 'Entity Bean

- Les méthodes “business”
 - correspond à l' interface distante

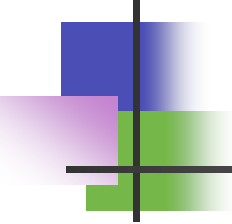
```
public double getBalance() throws RemoteException { return balance;}
public void setBalance(double d) throws RemoteException, AccountException{
    if (d<0) throw new AccountException(new AccountPK(this.branchno, this.accno));
    balance = d; }
public String getCustomer() throws RemoteException { return customer;}
public void credit(double d) throws RemoteException, AccountException{
    if (d<0) throw new AccountException(new AccountPK(this.branchno, this.accno));
    balance += d; }
public void debit(double d) throws RemoteException, AccountException{
    if (d<0 && (balance-d<0))
        throw new AccountException(new AccountPK(this.branchno, this.accno));
    balance -= d; }
```

...



Les méthodes de création

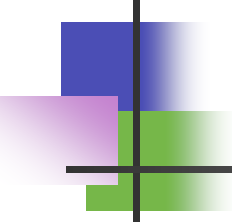
```
public void ejbCreate ( int branchno, int accno,  
                        String customer, double balance) throws  
        CreateException {  
    if(balance<0) throw new CreateException();  
    this.branchno=branchno;      this.accno=accno;  
    this.customer=customer;      this.balance=balance;  
}  
public void ejbPostCreate ( int branchno, int accno,  
                            String customer, double balance) { }  
}
```

Les méthodes de recherche

- sont générés au déploiement par le container

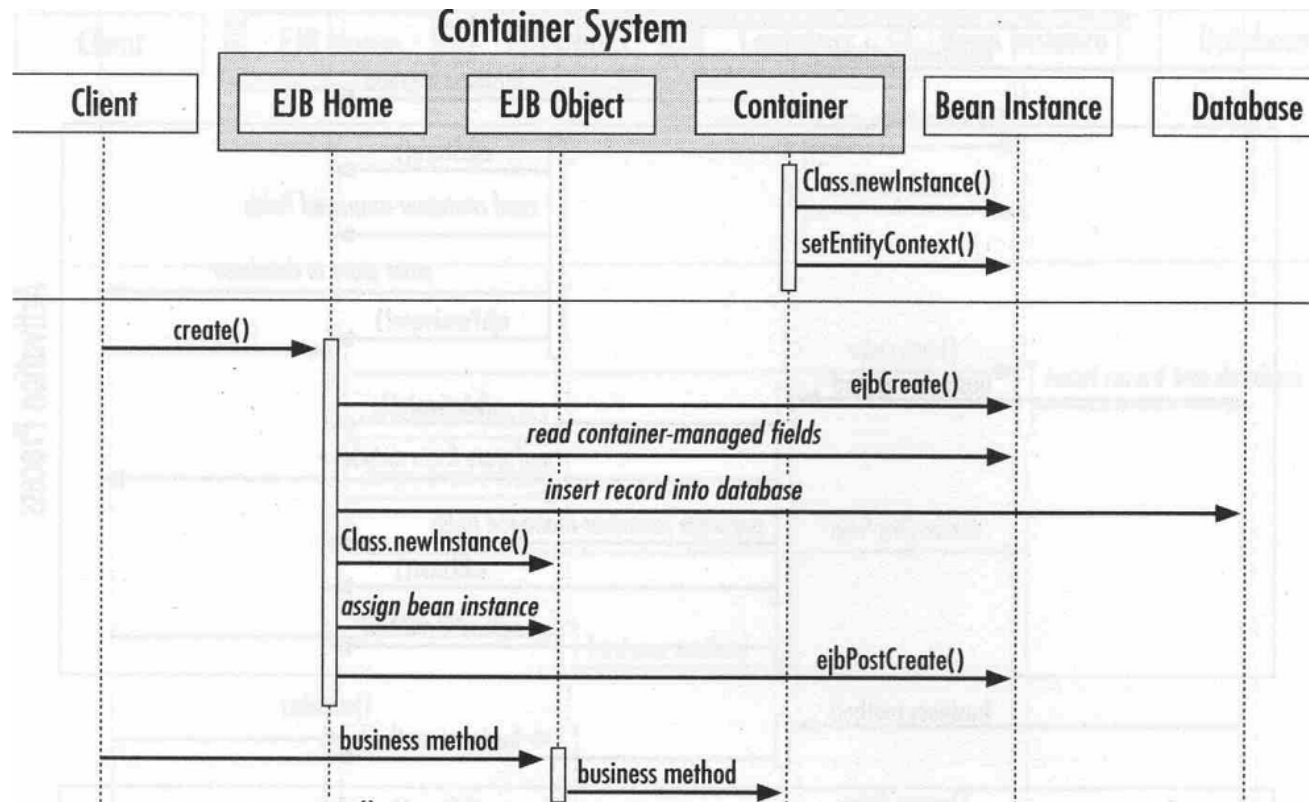
ejbFindByPrimaryKey (), **ejb**FindLargeAccounts (), **ejb**FindByCustomer ()

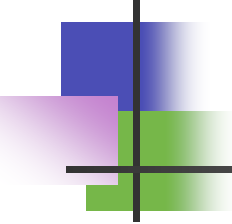


Remarque sur les méthodes de création

- pour chaque méthode `create(X)` de l'interface distante
- il y a un couple `ejbCreate(X)` et `ejbPostCreate(X)`
 - `ejbPostCreate(X)` effectue des compléments à la création

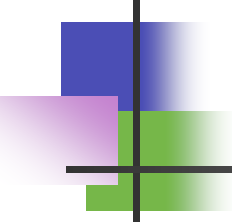
Remarque sur les méthodes de création





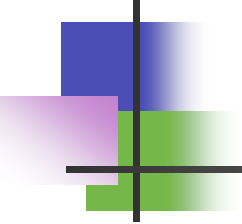
Les méthodes de synchronisation ejbLoad() et ejbStore()

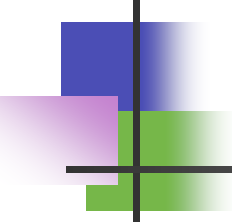
```
public class StringsBean implements EntityBean {  
    private transient EntityContext ctx;  
    private transient Vector msgvec;  
    public String    msgs; // Persistent field dans une colonne VARCHAR  
    public void ejbLoad() {  
        StringTokenizer st=new StringTokenizer(msgs,"~");  
        while(st.hasMoreTokens()) msgvec.addElement(st.nextToken());  
    } ... }
```



le lien avec la base de données

- `datasource.name jdbc_oracleI`
- `db.TableName Account`
- `db.Field.branchno branchid`
- `db.Field.accno accountid`
- `db.Field.customer customer`
- `db.Field.balance balance`

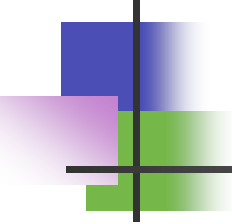
- 
-
- la méthode de recherche du bean
 - `db.Finder.findLargeAccount` where balance > ?
 - `db.Finder.findByName` where customer like ?



Exemple avec ejbCreate()

```
public void ejbCreate ( int branchno, int accno, String customer, double
    balance)
    throws CreateException {
    if(balance<0) throw new CreateException();
    this.branchno=branchno;      this.accno=accno;
    this.customer=customer;      this.balance=balance;
    Connection con = null; PreparedStatement ps = null;
    try { con = getConnection(); ps = con.prepareStatement(
        "insert into Account (branchid,accountid,customer,balance) values (?, ?, ?, ?)");
        ps.setInt(1, branchno);          ps.setInt(2, accno);
        ps.setString(3,customer);        ps.setDouble(4,balance);
    if (ps.executeUpdate() != 1) {
        throw new CreateException ("Failed to add Account to database");
    } catch (SQLException se) { throw new CreateException (se.getMessage());

    }
```

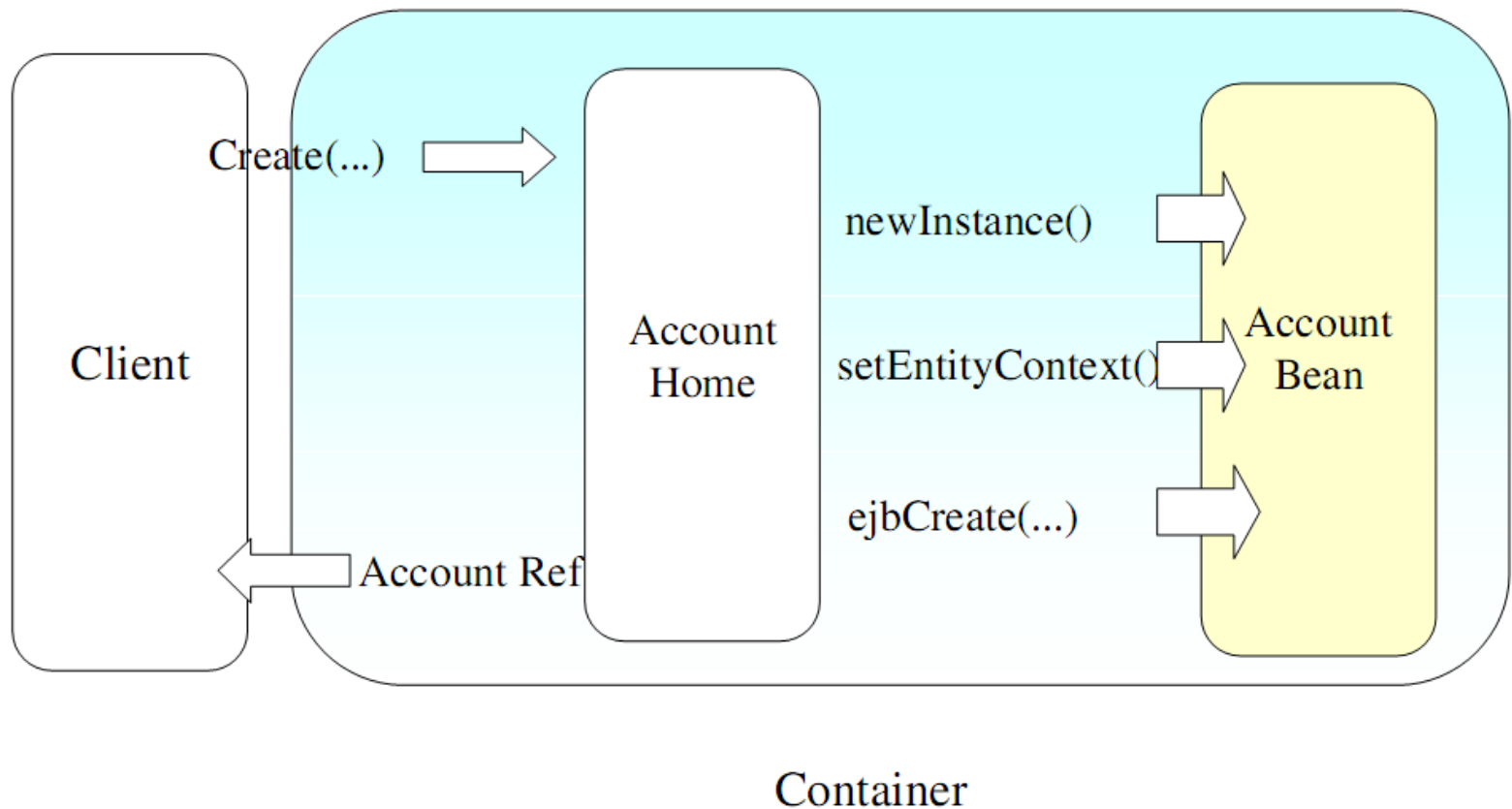


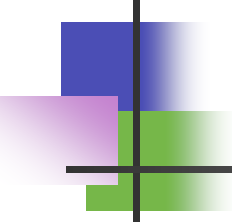
Exemple avec ejbLoad()

```
public void ejbLoad () throws RemoteException{
    AccountPK pk = (AccountPK) context.getPrimaryKey();
    Connection con = null; PreparedStatement ps = null; ResultSet result = null;
    try { con = getConnection(); ps = con.prepareStatement(
        "select customer,balance from Account where branchid=? and
        accountid=?");
        ps.setInt(1,pk.branchno); ps.setInt(2,pk.accno);
        result = ps.executeQuery();
        if(result.next()){
            branchno=pk.branchno; accno=pk.accno;
            customer = result.getString("customer");
            balance = result.getDouble("balance");
        }else{ throw new ObjectNotFoundException("Cannot find Account "+pk); }
    ...
}
```


Creation des instances

Home interface





Le client (Application Java)

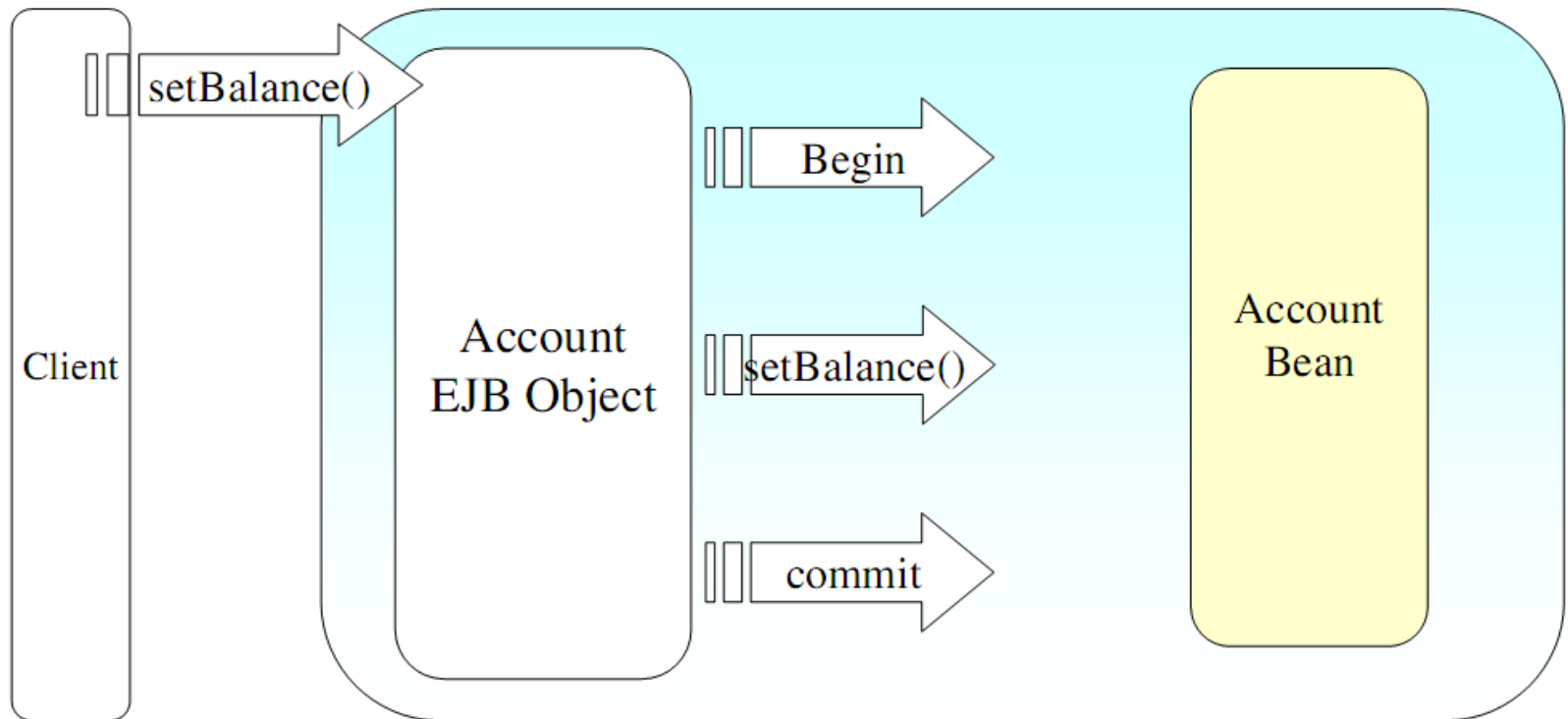
```
import bank.*; ...
public class ClientCreate{
    public static void main(String[] argv){
        AccountHome ah; Account a;
        try{ // rechercher l'interface EJB home interface
            InitialContext ctx = new InitialContext();
            Object objref = ctx.lookup("BankAccount");
            ah = (AccountHome)PortableRemoteObject.narrow(objref, AccountHome.class);
        } catch (NamingException) { NamingException.printStackTrace(); }
        // créer une instance d'entité
        a=ah.create(argv[0],argv[1],argv[2],0);
        // invoquer une méthode "business"
        System.err.println(a.getCustomer());
    }
}
```

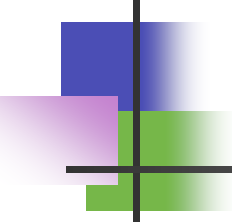
Le client (Application Java)

EJB1.1

```
import bank.*; ...
public class ClientCreate{
    public static void main(String[] argv){
        AccountHome ah; Account a;
        try{ // rechercher l'interface EJB home interface
            InitialContext ctx = new InitialContext();
            Object objref = ctx.lookup(" java : comp / env / e j b / BankAccount");
            ah = (AccountHome)PortableRemoteObject.narrow(objref, AccountHome.class);
        } catch (NamingException) { NamingException.printStackTrace(); }
        // créer une instance d'entité
        a=ah.create(argv[0],argv[1],argv[2],0);
        // invoquer une méthode "business"
        System.err.println(a.getCustomer());
    }
}
```

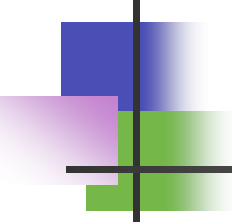
Remote interface





Le client (Application Java)

```
import bank.*; ...
public class ClientSetBalance{
    public static void main(String[] argv){
        AccountHome ah; Account a;
        try{ // rechercher l'interface EJB home interface
            InitialContext ctx = new InitialContext();
            Object objref = ctx.lookup("BankAccount");
            ah = (AccountHome)PortableRemoteObject.narrow(objref, AccountHome.class);
        } catch (NamingException) { NamingException.printStackTrace(); }
        // créer une instance d'entité
        try{
            a=ah.findByPrimaryKey(new AccountPK(argv[0],argv[1]));
        } catch (FinderException) { FinderException.printStackTrace(); }
        // invoquer une méthode "business"
        a.setBalance(argv[2]);
    }
}
```



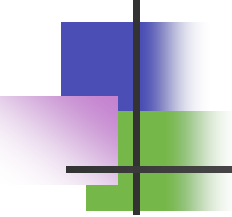
Le client (Servlet 1 / 2)

```
import ...

public class SetBalanceServlet extends HttpServlet {
    AccountHome ah;

    public void init(ServletConfig config) throws ServletException{
        try{
            InitialContext ctx = new InitialContext();
            Object objref = ctx.lookup("BankAccount");
            ah = (AccountHome)PortableRemoteObject.narrow(objref, AccountHome.class);
        } catch (NamingException) { NamingException.printStackTrace(); }
    }

    public void destroy() {
        System.out.println("Destroy");
    }
}
```



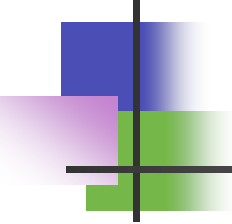
Le client (Servlet 2/2)

```
public void doGet (HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    response.setContentType("text/html");
    PrintWriter out = response.getWriter();
    out.println("<HTML><HEAD><TITLE>SetBalance</TITLE></HEAD><BODY>");
    int accno = (new Integer(request.getParameter("ACCNO")).intValue());
    int branchno = (new Integer(request.getParameter("BRANCHNO")).intValue());
    AccountPK pk=new AccountPK(branchno,accno);
    double newbalance=(new Double(request.getParameter("BALANCE")).doubleValue());
    try{
        Account a=ah.findByPrimaryKey(pk);                a.setBalance(newbalance);
        out.println("<H1>New Balance</H1>");
        out.println("<P>Account: " + pk + "<P><P>New Balance: " + a.getBalance() + "<P>");
    } catch (FinderException) { out.println("<H1>Problem</H1><P>No Account: " + pk + "<P>"); }
    out.println("</BODY></HTML>"); out.close();
}}
```

Descripteur de déploiement

Account
Deployment
Descriptor

Home interface	AccountHome
Remote interface	Account
Enterprise Bean	AccountBean
BeanHomeName	"BankAccount"
ControlDescriptors	TX_SUPPORTS
Env. properties	DataSource name ...
ContainerManagedFields	branchno, accno, customer, balance



Gestion déclarative des transactions

- Support du conteneur pour les transactions
 - le conteneur utilise les attributs de transaction pour réaliser la politique souhaitée
- • Attributs des transactions



TX_NOT_SUPPORTED
TX_NEVER
TX_REQUIRED
TX_SUPPORTS
TX_REQUIRES_NEW
TX_MANDATORY
TX_BEAN_MANAGED